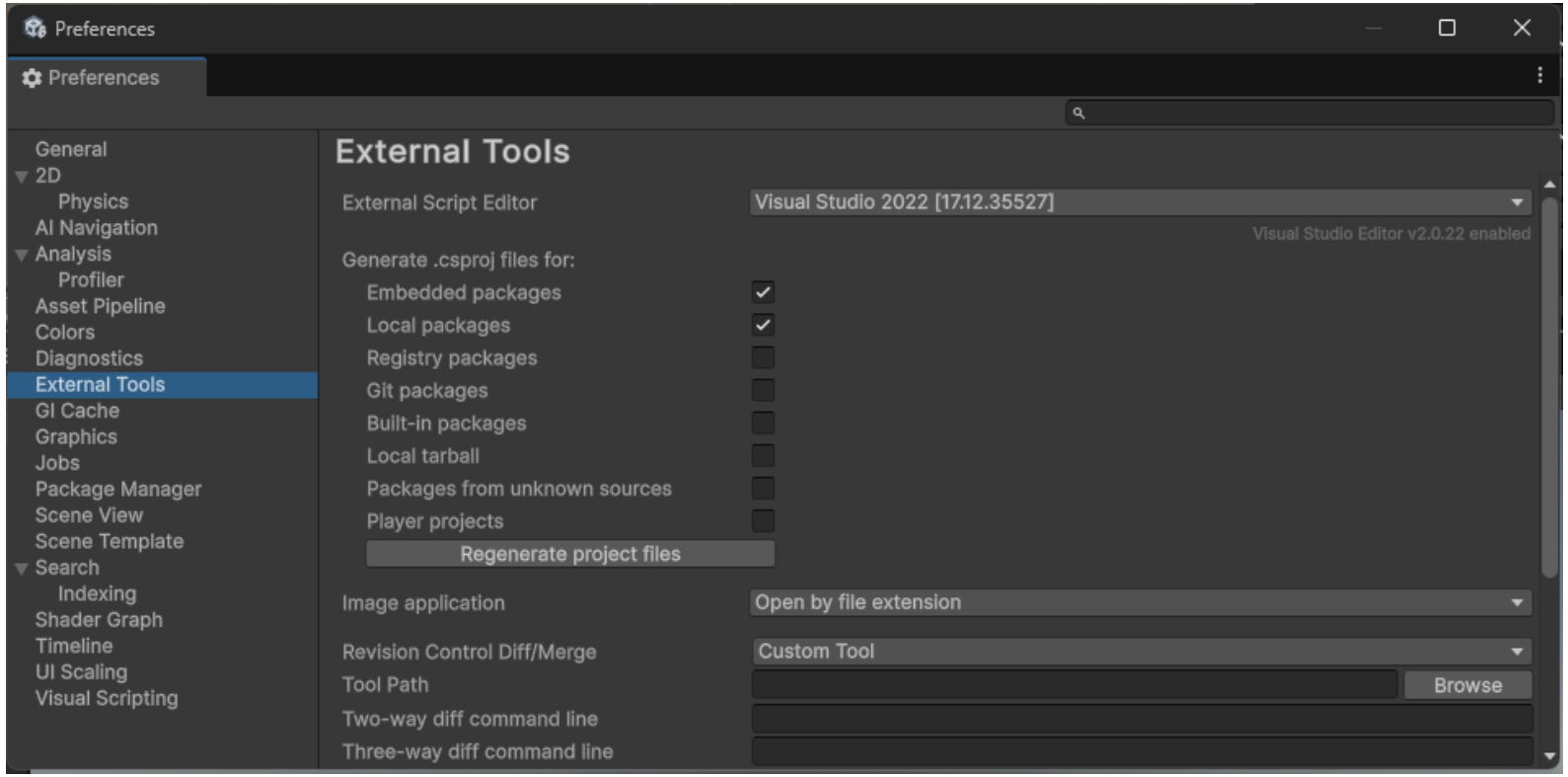

Shaded
2D
Unity

Unity – C# Bağlantısı

Bağlantı

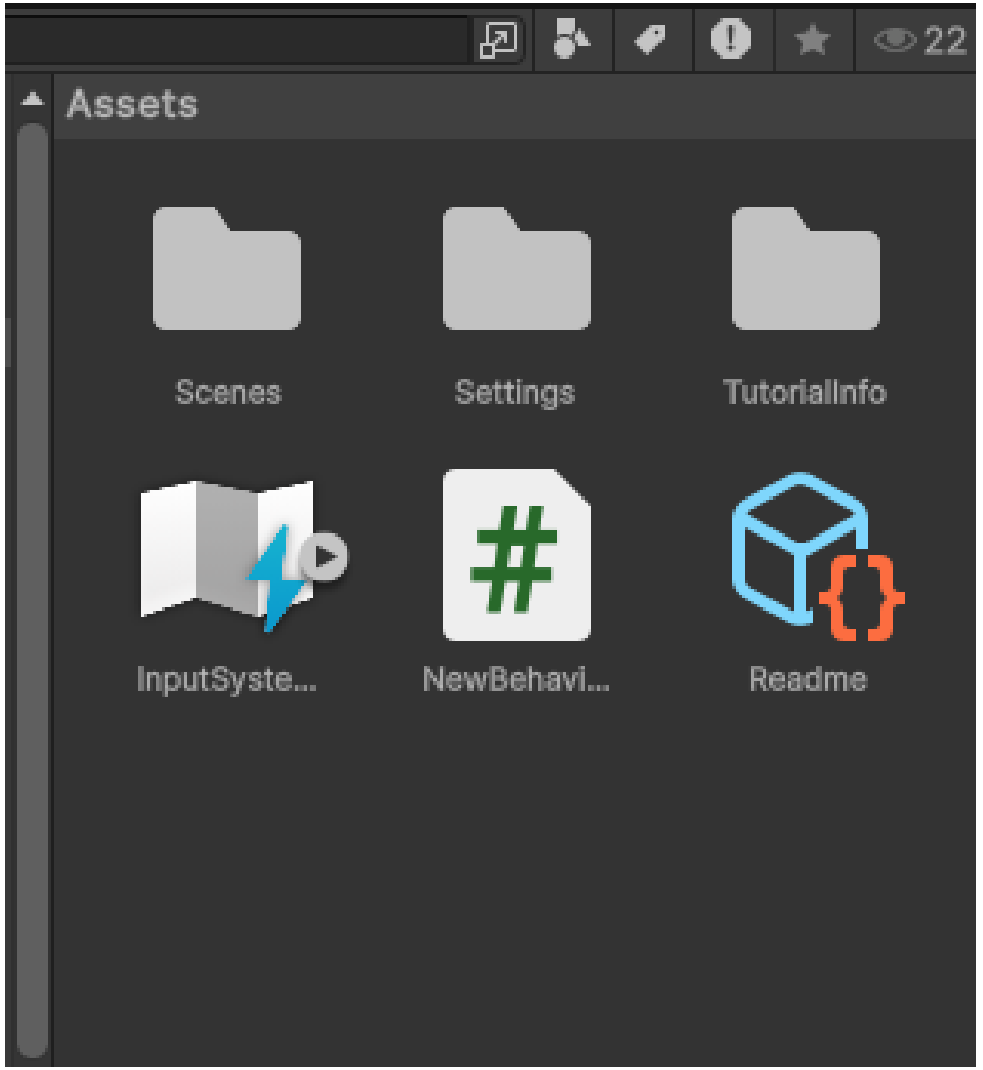
* Unity de script yazmak için visual studio ile bağlantı ayarlarının yapılması gerekir. Sistem otomatik olarak bunu yapar, yapmadığı durumlarda kontrol edilmesi gerekir.
* Edit → Preferences → External Tools → External Script Editor üzerinden seçim yapılır.



Script Oluşturma

Script Oluşturma

* Script oluşturmak için, scene de herhangi bir obje seçilip inspector ekranında add component diyerek new script seçilir ve script dosyasına bir ad verilir.
* Assets kısmında görünen script dosyası çift tıklanarak açılabilir.



Unity

Script Ekranı

Script İçerik

* Monoscript özellikli boş bir script dosyası start ve update fonksiyonlarını içerir
* Start fonksiyonu uygulama çalıştığı ilk an bir kere çalışan metottur.
* Update fonksiyonu uygulama çalıştığı andan itibaren her frame için çalışmaya devam eder

```
using UnityEngine;

Unity Script | 0 references
public class NewBehaviourScript : MonoBehaviour
{
    // Start is called once before the first execution of Update aft
    Unity Message | 0 references
    void Start()
    {


    }

    // Update is called once per frame
    Unity Message | 0 references
    void Update()
    {

    }
}
```

Ekran Yazdırma

Ekrana Yazdırma

* Debug.Log komutu ile consola istenilen şeyler yazdırılıp uygulama üzerindeki işlemler takip edilebilir

```
// Start is called once before the first execution of Update aft
Unity Message | 0 references
void Start()
{
    Debug.Log("Uygulama Başladı");
}

// Update is called once per frame
Unity Message | 0 references
void Update()
{
    Debug.Log("Her Frame de çalışıyor");
}
```

Sayaç Belirleme

Sayıcı

*

invokerepeatin(metodadi,başlama,bitiş)

```
Unity Message | 0 references
void Start()
{

    InvokeRepeating("logposition", 0f, 3f);
}


0 references
void logposition()
{
    Debug.Log("sayac");
}
```

Objeye Kuvvet Ekleme

Ekleme

* Herhangi bir objeye kuvvet eklemek için öncelikle o objenin ridigbody özelliğinin tanımlanması gerekir.
* Tanımlandıktan sonra script ekranında rigidbody değişkeni tanımlanır.
* Tanımlamadan sonra editor ekranında seçili objenin inspector kısmında script componentinde bu değişken görüntülenerek, objeyle ilişkilendirilmesi gerekir.
* Sonrasında Script ekranında addForce(x,y,z) eklenerek objeye hareket verilir.

```
public Rigidbody rb;
// Start is called once before the first execution of U
Unity Message | 0 references
void Start()
{
    rb.AddForce(0, 0, 200);
}

// Update is called once per frame
Unity Message | 0 references
void Update()
{

}
```

Objeyi Durdurma

Durdurma

- rb.angularvelocity = vector3.zero komutu ile birlikte herhangi bir objenizi açısal dönüşünü durdurabilirsiniz.
- Obje tam olarak durmaz açısal dönüşü durdurulur
- Rb.linearvelocity = vector3.zero tam olarak durdurur

```
public Rigidbody rb;
int i = 0;
// Start is called once before the first execution of Update after the Mon
Unity Message | 0 references
void Start()
{
    rb.AddForce(0, 0, 200);
}

// Update is called once per frame
Unity Message | 0 references
void Update()
{
    i++;
    if(i == 300)
    {
        |
        rb.linearVelocity = Vector3.zero;
    }
}
```

Konuma Göre işlemler

Konum tanıma

- Transform.position.eksen bilgisi o objenin konumuna göre istenilen işlemleri yapmayı sağlar.

```
// Update is called once per frame
Unity Message | 0 references
void Update()
{
    if(transform.position.z > 3 && transform.position.z < 3.2)
    {
        rb.AddForce(10, 10, 0);
    }
}
```